



**National University of Computer & Emerging Sciences, Karachi**  
**Spring – 2026 School of Computing (BSCS)**  
**Assignment 2**



**Subject: (CS3009) Software Engineering**

**Post Date: 30/3/2026**

**Total Marks: 50**

**Due Date: 15/4/2026, 11:59 PM**

**Course Instructors: Sobia Iftikhar, Hajra Ahmed, Shaheer Ahmed, Engr Abdul Rahman, Ahmed Qaiser**

**Question 1:**

MediTrack is a new web-based patient portal being developed for a large hospital network, designed to provide patients with a comprehensive digital platform for managing their healthcare needs. The system will enable patients to view their medical records and test results, schedule and manage appointments, securely communicate with healthcare providers, request prescription refills, and complete pre-visit questionnaires. The hospital anticipates serving over 15,000 daily active patients once the system is fully operational. Given the sensitive nature of medical information, the system must comply with strict healthcare data privacy regulations, including HIPAA. The requirements for the system are only partially known at the outset, with additional needs expected to emerge through ongoing stakeholder feedback throughout the development process. The hospital's IT department possesses experience with both waterfall and agile development methodologies, providing flexibility in choosing an appropriate process model. The development team consists of 12 members, including eight developers, two UI designers, and two testers. The project timeline requires that the first usable version be delivered within four months, with the complete system to be fully deployed within ten months.

Answer the following:

1. Which software process model would you recommend for MediTrack? Justify your choice with four specific reasons based on the project characteristics described above.
2. If you recommended a hybrid approach, explain which parts of the system would follow a plan-driven approach and which would follow an agile approach. If you recommended a pure model, explain why a pure approach is sufficient
3. Define abstraction and information hiding as design concepts. For each, provide one concrete example from the MediTrack system, showing how it would be implemented.
4. Explain the relationship between cohesion and coupling. In the context of MediTrack, describe one scenario where high cohesion would be beneficial and one scenario where low coupling would be beneficial.

**Question 2:**

A university is developing a Smart Campus System, a comprehensive, integrated platform designed to unify and streamline various campus services. This ambitious system will encompass five core subsystems: Course Management for handling student registration, grade management,

and class schedules; Facility Management to oversee classroom booking, room availability tracking, and maintenance requests; Security & Access for building access control and security camera monitoring; Student Services to manage financial aid, housing assignments, and meal plans; and Emergency Notification for delivering campus-wide alerts during critical situations.

The system must support a large-scale user base of over 50,000 students and 5,000 faculty and staff members, requiring robust architectural design to handle this volume. Critical services within the system demand 24/7 high availability, as any downtime could disrupt essential campus operations. The Security & Access subsystem presents unique technical challenges, as security cameras generate continuous data streams that require real-time processing for motion detection and threat identification. The Emergency Notification subsystem imposes strict performance constraints, with emergency alerts required to reach all users within 30 seconds of activation.

Requirements:

- User Scale: 50,000+ students and 5,000+ faculty/staff must be supported
  - Availability: High availability (24/7) required for critical services
  - Data Processing: Security cameras generate continuous data streams requiring real-time processing
  - Performance: Emergency alerts must reach all users within 30 seconds
  - Access Control: Different user roles (students, faculty, administrators, security personnel) have varying access levels
  - Integration: System must integrate with existing legacy databases
  - Evolvability: New features will be added overtime throughout the system lifecycle
1. Recommend the most suitable architectural pattern from those covered in class.
  2. The overall Smart Campus System must be available 24/7. Explain two architectural decisions you would make to ensure high availability. For each decision, explain which system characteristic (performance, security, safety, availability, maintainability) it addresses.
  3. Draw a detailed diagram that should clearly show:
    - Model components (data structures, database interactions)
    - View components (different user interfaces for students vs. faculty)
    - Controller components (business logic, validation)
    - Arrows indicating data flow and control flow
  4. Explain the concept of refactoring in software design. Provide one specific example of how refactoring might be applied to the Smart Campus System during its evolution.
  5. The Smart Campus System will have a long lifespan (10+ years). Explain why modularity is particularly important for this system and how it supports maintainability over time.

### **Question 3:**

#### **Case Study: Smart Expense Sharing System (Like a Split wise app)**

Design and implement a mini expense-sharing platform where users can manage shared expenses within groups.

For example: You are building a system like an app on play store “Splitwise”.

3 friends: Ali, Sara, Ahmed

- Ali pays 3000 for dinner (split equally)
- Sara pays 1500 for fuel

System should calculate: Who owes whom? And show Final balances

#### ***Core Features:***

Implement a backend system (CLI based or basic UI, either is acceptable) having following modules and sub-components:

1. User Management (Create user, Add multiple contact numbers per user, Unique email validation)
2. Group Management (Create group, Add/remove members, Assign admin roles)
3. Expense Management (Add expense (equal, exact, percentage split))
4. Balance Calculation Engine
  - Shows Who owes whom and Net balances
  - Edge cases could be Circular debts and Partial settlements
5. Settlement Module Record payments and Update balances dynamically

#### **Part 1: Code Development**

You are required to Write clean and modular code (OOP preferred). Implement at least 3 complex algorithms, e.g.: Correctly split expenses, Track balances, simplify debts (reduce number of transactions) or something else if you want.

#### **Part 2: Manual Test Case Design**

You are required to design comprehensive manual test cases. Required test coverage should be from Functional Testing (Valid/invalid expense splits, Adding/removing users in groups), Boundary Testing (Zero/negative expenses), Equivalence Partitioning (Valid vs invalid input classes). Expected Format for writing manual testcases should be:

| Testcase ID | Description | Input | Expected Output | Observed Output | status |
|-------------|-------------|-------|-----------------|-----------------|--------|
|-------------|-------------|-------|-----------------|-----------------|--------|

#### **Part 3: Automated Testing**

You are required to use any framework and write at least 10 automated test cases including unit and integration tests: for example:

1. Python: unittest / pytest
2. Java: JUnit
3. JS: Jest

#### Part 4: Test Report

You are required to explain:

1. your approach and understanding of the problem statement:
2. Why specific test cases were chosen
3. Challenges in testing logic
4. Comparison: Manual vs Automated testing